# DBdoctor: A Fine-grained and Non-intrusive Performance Diagnosis Platform for Databases

Xinyue Shi†, Quanqi Xin‡, Zhengjin Wang†, Xinyi Zhang†, Haoqiong Bian†, Wei Lu†
Qiyu Zhuang†, Shuang Liu†, Jikuan Zhang‡, Xiang Zheng‡, Yunpeng Chai†, Xiaoyong Du†
† *Renmin University of China* ‡ *Juhaokan Technology, Hisense*

†{xinyueshi, zhengjin.wang, xinyizhang.info, bianhq, lu-wei, qyzhuang, shuang.liu, ypchai, duyong}@ruc.edu.cn
‡{xinquanqi, zhangjikuan, zhengxiang2}@hisense.com

***Abstract*—Database performance anomalies are prevalent in large-scale deployments and often result in substantial business losses, making online diagnosis and resolution of anomalies indispensable in production environments. Existing anomaly diagnosis frameworks rely on sample-based external monitoring tools, which suffer from two critical limitations: (1) an unfavorable granularity–overhead trade-off, where low-frequency sampling misses transient anomalies while high-frequency sampling introduces significant overhead, and (2) black-box inference due to a lack of internal database context. To address these limitations, we propose** DBdoctor**, a non-intrusive performance diagnosis platform applicable across different database engines.** DBdoctor **proposes a novel event-based metric collection framework that leverages eBPF to collect fine-grained both external and internal database metrics with low overhead. These metrics are modeled as SQL temporal resource metrics and dependency graphs to enable white-box anomaly diagnosis, supporting precise root cause identification.** DBdoctor **has been deployed in large enterprises such as China Unicom and Hisense to manage thousands of database instances. Experimental evaluations using popular benchmarks and real-world workloads demonstrate that** DBdoctor **achieves higher diagnosis accuracy than existing approaches, with comparable or lower performance overhead.**

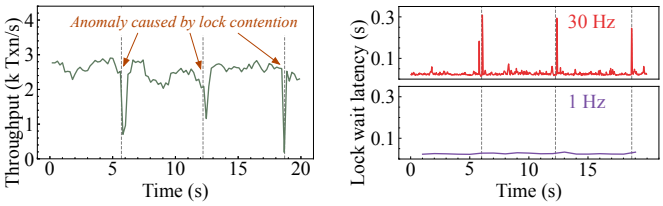
***Index Terms*—database monitoring, anomaly diagnosis, eBPF**

## I. Introduction

As database deployments scale, the performance anomalies arising from unpredictable workload patterns, resource contention, inefficient SQL statements, or hardware failures have become an inevitable challenge. Such anomalies can cause serious availability issues, including SLA violations and service outages, making effective anomaly diagnosis a critical capability for modern database systems [24].

In production environments, the diagnosis process is typically conducted online and involves three phases: (1) *Metric collection* that gathers predefined metrics related to performance anomalies, such as CPU utilization; (2) *Root cause identification* that analyzes the collected metrics to identified anomalies and their root causes (e.g., missing indexes), either manually by DBAs or automatically using machine learning methods; and (3) *Anomaly resolution* that suggests and applies targeted optimizations to mitigate the performance anomalies. Of the three phases, metric collection provides inputs for the subsequent phases and fundamentally impacts the efficiency and accuracy of diagnosis. Existing monitoring tools, either general-purpose (e.g., Prometheus [17], perf [14]) or DBMS-native (e.g., pg_stat_statements [16] for PostgreSQL), typically adopt a *sample-based external* monitoring approach [14], [16], [17] where predefined metrics are collected at fixed intervals (e.g., every 1 s). However, such a monitoring approach introduces two major limitations.

†Haoqiong Bian, ‡Xiang Zheng, ‡Jikuan Zhang are corresponding authors.




(a) TPS degradation anomalies (b) Sampled lock-wait metrics

Fig. 1: Anomalies and effects of various sampling frequencies.

**L1. Unfavorable granularity–overhead trade-off.** The sample-based monitoring paradigm periodically collects metrics from the predefined OS or database interfaces. While this approach is simple and general, its sampling frequency directly trades off accuracy against overhead. To avoid runtime interference, production systems typically adopt relatively low sampling frequencies (around 1 Hz or lower), which inevitably risks missing critical but short-lived root cause events. To illustrate, we use YCSB [22] on MySQL to simulate a high-concurrency scenario and periodically inject a heavy range-scan query, simulating a user browsing request. As shown in Figure 1a, this query temporarily monopolizes locks and blocks other concurrent transactions for about 150 ms, causing transaction throughput drops. As shown in Figure 1b, 30 Hz monitoring successfully captures the lock-contention spikes but incurs considerable runtime interference (8.72% throughput degradation). In contrast, 1 Hz monitoring introduces negligible overhead but entirely misses the contention spikes. Prior studies show that even a 100 ms increase in web response latency can reduce sales by 1% and decrease conversion rates by 7% [1], [2]. Persistently overlooking such transient anomalies leaves many performance issues undiagnosed. In complex production environments, their effects can cascade across components, leading to significant business losses. *This motivates the need for a fine-grained, low-overhead monitor-*

*ing framework to capture lossless critical root-cause events.*

**L2. Black-box inference due to lack of internal context.** Existing diagnosis methods operate primarily on the system's external outputs and lack visibility into the internal execution contexts of databases. Some of them [26], [35] perform diagnosis at the *system level*, relying on coarse-grained metrics like CPU utilization or overall transaction throughput. Others [15], [18], [29], [41] attempt to diagnose at the *query level*, obtaining per-query wait event statistics through database-specific plugins or system tables, tightly coupled to particular database implementations. However, all existing methods provide only external or aggregated statistics, failing to capture a query's *full execution lifecycle* or its *interactions with other concurrent queries*. Both aspects are crucial for accurate root cause analysis: understanding the execution lifecycle reveals where performance bottlenecks arise in query runtime (e.g., which actions of which queries contribute most to anomalous CPU spikes), whereas modeling inter-query dependencies uncovers how anomalies propagate through lock waits. The inability of existing methods to capture these aspects stems from the lack of *internal observability*—the capability to correlate higher-level SQL activities with low-level runtime events (e.g., resource consumption, lock operations). As a result, they can only observe system symptoms but cannot causally link them to query behaviors, thereby treating the database as a black box and failing to accurately localize root causes. We present a case study in §II-A to further explain these limitations. *This motivates the need for a monitoring framework that provides internal observability and bridges query-level behaviors and function-level runtime events to enable white-box diagnosis.*

To address these limitations, we propose DBdoctor, a novel diagnosis platform for DBMS empowered by event-based, fine-grained, and internal metric collection with low overhead. Instead of relying on fixed sampling frequencies, DBdoctor attaches probes to specific database functions and triggers metric collection upon predefined events, ensuring lightweight and lossless monitoring. To achieve this, we adopt extended Berkeley Packet Filter (eBPF) [8], a lightweight in-kernel tracing and instrumentation framework that allows user-defined programs to be safely executed in the Linux kernel and attached to user or kernel functions for metric collection.

Although eBPF enables non-intrusive and low-overhead metric collection by bypassing the user space and executing efficiently in the kernel, applying it for database monitoring presents two major challenges. First, capturing lossless anomaly-critical events with low runtime overhead. Leveraging eBPF, DBdoctor can achieve lossless metric collection by probing database and kernel functions and collecting metrics at each function invocation. However, naively monitoring all functions and reporting every invocation, even with eBPF, can incur excessive runtime overhead. Second, transforming raw eBPF messages into SQL-level metrics efficiently. Metrics collected by eBPF are inherently function-level and lack database context, making it difficult to associate them with SQL queries. Under heavy workloads, substantial events trigger massive eBPF message generation. Efficiently managing these kernel-space messages and constructing SQL-level metrics from the raw eBPF data is highly challenging. To overcome these challenges, DBdoctor introduces two components:

(1) *Adaptive event-based metric collection framework.* We propose an event-based, fine-grained metric collection paradigm to resolve the granularity-overhead trade-off inherent in existing sampling-based approaches. Instead of periodically sampling external metrics, DBdoctor attaches eBPF probes to critical database and kernel functions, collecting fine-grained internal metrics per function invocation. To reduce metric collection overhead, we present an offline probe set identification approach to only inject probes into a high-impact set of functions necessary for effective anomaly diagnosis. Moreover, we design an online anomaly-aware probing mechanism that prioritizes the full capture of anomalous events while efficiently aggregating non-anomalous metric data of continuous events into a single message, ensuring both lossless capture of anomaly events and minimal runtime overhead.

(2) *White-box anomaly diagnosis component.* We provide internal database observability for anomaly diagnosis by modeling raw eBPF messages into SQL temporal resource metrics and SQL dependency graphs. To achieve this, we correlate the collected function-level metrics with high-level SQL statements using correlation identifiers such as query ID and thread ID. With both external and internal fine-grained metric data, DBdoctor performs white-box anomaly diagnosis, identifying precise root causes and performing effective resolutions.

We summarize our contributions as follows.

- We propose an adaptive event-based metric collection framework that leverages eBPF to collect fine-grained database metrics with low overhead. It is non-intrusive and does not depend on a specific DBMS implementation.
- We propose a white-box anomaly diagnosis approach that captures the function-level database internal context to construct SQL temporal resource metrics and dependency graphs, enabling precise diagnosis.
- We implement these approaches in a production diagnosis platform, namely DBdoctor, which has been deployed in over 500 enterprises, including China Unicom, BMW, and Hisense, to monitor and diagnose dozens to thousands of database instances, heterogeneous or not.
- We evaluate DBdoctor's diagnosis performance using real-world workloads in popular databases. The result shows that DBdoctor outperforms baselines in root-cause identification accuracy with comparable or lower overhead.

## II. Preliminary

### A. Case Study for Performance Diagnosis

Database performance anomalies often exhibit *transient*, *propagated*, and *compounded* characteristics, which fundamentally challenge existing monitoring and diagnosis approaches. These properties motivate the design of a monitoring framework that captures fine-grained and internal query execution behaviors. We have illustrated the transient nature of anomalies using Figure 1. In this subsection, we present

a multi-root-cause anomaly case to further demonstrate the necessity of internal execution observability. In the DBMS, a large range scan holds long-lived locks, blocking numerous concurrent transactions and inducing widespread lock contention. At the same time, a set of redundant indexes continuously consumes CPU and memory resources during query execution and maintenance. Together, these issues result in hundreds of queries running more than 200% slower than their historical baselines, accompanied by continuous CPU and memory spikes. However, existing diagnosis approaches struggle to accurately and efficiently identify all root causes, as they manifest similar external symptoms—resource exhaustion and a large number of slow queries. Although query-level metrics (e.g., numerous LCK_M_IU wait events observed in slow queries) are provided, they remain aggregated and causality-agnostic: they do not reveal which SQL statements and which execution actions are responsible for the blocking and resource exhaustion. Consequently, distinct root causes that manifest similar symptoms become difficult to identify, especially when multiple interacting root causes coexist.

This example highlights that effective diagnosis requires internal observability with two critical dimensions. First, monitoring tools must expose the *dynamic global dependencies* among concurrent queries. In this example, existing tools provide only isolated snapshots of blocking relationships (e.g., one transaction waiting for another), forcing diagnosis systems to repeatedly query system tables (e.g., *pg_locks*) and manually reconstruct dependency chains across hundreds of queries, which is inefficient and error-prone. Second, monitoring tools must establish a *direct association between anomalies and SQL execution actions*. In this example, identifying all root causes require to know which SQLs are executing which actions and consume which resources at the time when CPU and memory spikes occur. However, existing tools expose only aggregated statistics, such as cumulative wait-event counts or total resource usage, obscuring these critical causal relationships. As a result, existing diagnosis approaches largely rely on historical experience or pattern matching over aggregated metrics, which is inherently inadequate for accurate and efficient root-cause diagnosis.

### B. eBPF Technology

eBPF (extended Berkeley Packet Filter) is a kernel-native technology that allows user-defined programs to be safely executed within the OS kernel without kernel modifications [8]. An eBPF program is typically written in a restricted C-like language, compiled into eBPF bytecode, and loaded into the kernel via dedicated system calls (e.g., bpf() syscall). Before execution, the kernel performs static verification to ensure safety properties such as memory safety and bounded loops. Once verified, the bytecode is just-in-time (JIT) compiled into native instructions and attached to specific hook points, such as system calls, kernel functions (kprobes), user functions (uprobes), or tracepoints. The program is then triggered whenever the corresponding kernel or user-space event occurs, enabling fine-grained, event-driven monitoring of kernel and application behaviors, while ensuring system stability and security.

**eBPF communication mechanism.** eBPF provides several mechanisms for exposing kernel-collected information to user programs. (1) Perf Buffer. The perf buffer is a per-CPU circular buffer that enables efficient data exchange between eBPF programs and the user space. (2) Ring buffer. It is a multi-producer, single-consumer circular buffer enabling communications between kernel and user space. (3) eBPF Maps. It can store various data structures, designed for storing and sharing data between different eBPF programs and between eBPF programs and user-space applications. (4) Trace Pipe. For debugging or prototyping, eBPF programs can write messages into the *trace_pipe* file under *debugfs*.

## III. System Overview

DBdoctor works as an anomaly diagnosis and resolution platform for databases. It follows a non-intrusive and vendor-agnostic diagram, making it universally applicable across different database engines. This is critical for production environments as large organizations often have heterogeneous databases for various applications. DBdoctor can serve as a unified, standardized platform for diagnosing different database systems, avoiding the management and operational complexity of heterogeneous diagnostic ecosystems.

DBdoctor attaches eBPF programs (referred to as eBPF probes) to critical database functions and kernel functions to track database events and resource consumption in real-time, and models database context as SQL temporal resource metrics and SQL dependency graphs. Thus, it can accurately diagnose the root causes of anomalies and provide actionable resolutions. As shown in Figure 2, the platform consists of four core components. (1) *Probe Manager* maintains the optimal eBPF probe set for each database. It dynamically injects or removes probes on demand, ensuring fine-grained metric collection at runtime with low overhead. (2) *Event Stream Processor* efficiently reads the messages generated by the probes from kernel space into user space, and forwards these messages into a Kafka message queue. (3) *Database Context Reconstructor* consumes raw eBPF messages and reconstructs them into SQL temporal resource metrics and SQL dependency graphs. (4) *Anomaly Analyzer* identifies performance anomalies, analyzes root causes, and provides targeted solutions.

**Adaptive probing.** This phase aims to balance the monitoring precision and overhead. In this phase, before monitoring a target database, the *Probe Manager* performs offline probe set selection that uses a utility-aware probe selection method to determine an optimal function set for injection under a preferred monitoring overhead constraint (§IV-A). The *Probe Manager* stores the optimal probe set for each database version in a JSON configuration file, enabling efficient reuse across subsequent monitoring of the same or similar DBMSs. By default, only functions in the optimal probe set are injected. At runtime, when a fine-grained analysis of specific metrics is required, *Probe Manager* supports on-demand probe injection,

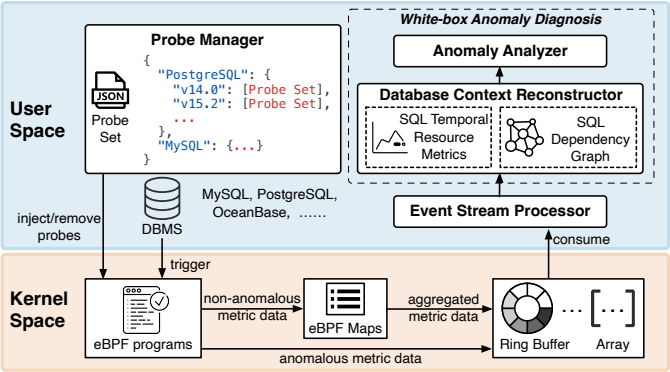



Fig. 2: Overview of DBdoctor.

enabling additional probes to be dynamically attached to relevant functions without interrupting database execution.

**Anomaly-aware message generation.** In this phase, the injected eBPF probes are triggered to capture database internal contexts whenever their associated database functions are invoked. DBdoctor employs an anomaly-aware message generation strategy that adaptively controls the frequency of eBPF message emission (§IV-B). Upon activation, each probe executes a predefined callback function that collects metric data such as thread ID, query ID, transaction ID, resource usage, and lock states. The collected data is buffered in eBPF maps and then aggregated into eBPF messages adaptively. Specifically, kprobes injected into kernel functions generate resource-consumption messages, whereas uprobes injected into database functions generate database event messages. This entire process occurs in kernel space without requiring user-space intervention. On the user side, the *Event Stream Processor* continuously consumes messages from the ring buffer and eBPF Map data structures, and forwards them into a Kafka message queue [27] for subsequent processing.

**Database internal context modeling.** In this phase, the *Database Context Reconstructor* consumes messages from the Kafka message queue, modeling the database internal context as SQL temporal resource metrics (§V-A) and a temporal SQL dependency graph (§V-B). The raw messages are first indexed by their query ID and then categorized by resource dimension, including CPU, I/O, memory, network, and lock counts. For each SQL, messages are converted into a temporal resource-metric sequence embedded with query-execution semantics, reflecting resource consumption throughout the query's execution lifecycle. These per-query metrics are further aggregated by SQL template, where all executions of the same template contribute to a unified template-level resource metric. Meanwhile, raw lock event messages are consumed to construct a temporal SQL dependency graph that records the evolving lock relationships among concurrent queries.

**White-box anomaly diagnosis.** In this phase, *Anomaly Analyzer* performs automated detection, white-box diagnosis, and targeted optimization. Anomalies are detected when either system-wide metrics or the metric patterns of individual SQL templates deviate significantly from normal behavior (§V-C1). Once an anomaly is detected, the system extracts the temporal resource metrics of all SQL templates and the SQL dependency graph within the anomaly time window. The *Anomaly Analyzer* identifies root causes and root-cause SQLs using a combination of feature-based anomaly detection and correlation analysis (§V-C2). After that, it proposes resolution strategies tailored to the identified root causes.

**Running Example.** We use the anomaly case in Figure 1 to illustrate the workflow. The *Probe Manager* first selects the optimal probe set offline and injects probes when the DBMS starts. For example, a probe in the set is attached to the function *LockAcquire()* to diagnose lock-related anomalies. During transaction execution, each invocation of *LockAcquire()* triggers the probe and enters the message generation pipeline. During normal execution (i.e., when no full-table scan occurs), multiple probe invocations triggered by *LockAcquire()* are aggregated into a message to reduce overhead. When a full-table scan is executed, the probe generates an individual message per invocation, as these invocations are identified as anomaly-critical (e.g., the lock waiting time exceeds a predefined threshold). Each message records contextual information, such as the thread ID, lock wait duration, and the identifier of the locked row. The generated messages are processed by the *Reconstructor*, which extracts the thread ID from each message and correlates it with the corresponding query ID previously captured at the SQL entry function (e.g., *exec_simple_query()*). Based on this correlation, DBdoctor reconstructs SQL-level temporal resource metrics and incrementally updates the dependency graph to capture blocking relationships among concurrent queries. Finally, the *Anomaly Analyzer* detects abnormal spikes in the lock-related metric and traverses the temporal dependency graph to accurately identify the full-table scan query as the root-cause SQL.

## IV. Metric Probing and Collection

The adaptive event-based metric collection framework in DBdoctor is crucial for precise anomaly diagnosis and resolution. In this section, we explain how this framework identifies the optimal function set to probe (§IV-A) and how the probes are dynamically attached to database instances to collect fine-grained metrics for diagnosis with low overhead (§IV-B).

**Message Types.** In DBdoctor, we package metric data into eBPF messages and collect two types of messages: (1) resource usage messages that are collected through the kprobes attached to OS kernel functions. They capture the real-time consumption of CPU, memory, I/O, and network resources; (2) database event messages that are collected through uprobes attached to database functions. They capture the SQL actions and background database events. For example, a *lock_wait* event represents that a SQL is waiting for locks, and a *log_flush* event represents that the database is flushing logs to disk. Both message types embed correlation identifiers (e.g., SQL ID, process ID, thread ID, or file descriptor), allowing the observed resource usage to be linked to database events. For example, the I/O usage messages can be correlated with SQL actions via the corresponding file descriptors.

### A. Offline Probe Set Identification

Probing all relevant kernel and database functions can achieve comprehensive and fine-grained monitoring. However, this introduces significant performance overhead even with eBPF. Therefore, we propose a probe set identification mechanism to select the most effective set of functions to probe.

Resource usage is collected via kprobes attached to kernel functions. For instance, probes on *vfs_read()* and *vfs_write()* capture I/O costs, while *tcp_sendmsg()* monitors network usage. These probes are indispensable as they provide the foundation of resource monitoring. Fortunately, the number of such kernel functions is relatively small, and each probe extracts metrics in constant time, resulting in low overhead.

In contrast, database event messages are gathered by attaching uprobes to the database's internal functions. Since probing every function would incur prohibitive overhead, we design a utility-aware probe-selection method to determine the optimal function set $\mathcal{F}_{opt}$ for database event monitoring, thereby minimizing probing cost while preserving diagnostic coverage. The selection process consists of two steps: base function set construction and probe set refinement.

**Base Function Set Construction.** We first pre-select candidate functions from the symbol table of the DBMS kernel [1] to construct a minimal but complete function set $\mathcal{F}_{base}$ that ensures full observability for SQL temporal resource metric construction and dependency graph generation. We categorize database events into three types: (1) resource consumption events, $\mathcal{F}_{res}$, which capture SQL actions that consume CPU, I/O, memory, and network resources; (2) SQL dependency events, $\mathcal{F}_{dep}$, which capture query blocking relationships such as lock acquisition and release; (3) background events, $\mathcal{F}_{back}$, which capture resource-consuming background activities such as flushing or purging logs. By combining expert knowledge and symbol table analysis, we identify the basic database functions of these types, forming the base set $\mathcal{F}_{base}$:

$$\mathcal{F}_{base} = \mathcal{F}_{res} \cup \mathcal{F}_{dep} \cup \mathcal{F}_{back} \tag{1}$$

To minimize monitoring overhead, we exclude low-level functions that lack query execution semantics (e.g., *memcpy()*) and functions that are excessively invoked during SQL execution and covered by higher-level functions (e.g., *ReadRecord()*).

**Probe Set Refinement.** Then we refine $\mathcal{F}_{base}$ into an optimal probe set $\mathcal{F}_{opt}$ through quantitative evaluation of each function's *diagnostic utility* $\mathcal{U}(f)$, which measures the importance of adding a probe to function $f$ for anomaly diagnosis. As defined in Eq.2, it combines two components: (1) the anomaly utility $\mathcal{U}_{ano}(f)$, reflecting the function's sensitivity to anomalies, and (2) the expert prior weight $\mathcal{W}(f)$, representing the domain knowledge on its diagnostic importance.

$$\mathcal{U}(f) = max(\ \mathcal{U}_{ano}(f),\ \mathcal{W}(f)\ ) \tag{2}$$

$\mathcal{U}_{ano}(f)$ is calculated as the difference in resource consumption between the normal and anomalous states. Specifically, we collect a set of real-world anomaly cases $\mathcal{M}$, which captures prevalent performance issues that are independent of specific workloads, and conduct controlled experiments under two conditions: (1) a normal baseline state $S_{norm}$ and (2) an anomalous state $S_m$ where anomaly case $m \in \mathcal{M}$ is injected. For each resource dimension $res \in \mathcal{RS}$ and function $f$, we denote the resource consumption of $f$ in system state $S$ as $T_{res}(f, S)$, where $\mathcal{RS}$ denotes the set of resource dimensions considered. The corresponding resource increment is defined as $\Delta T_{res}(f, S_m) = T_{res}(f, S_m) - T_{res}(f, S_{norm})$. Then, $\mathcal{U}_{ano}(f)$ is calculated as the aggregated normalized increment across all resources and anomaly cases, as defined in Eq.3.

$$\mathcal{U}_{ano}(f) = \frac{1}{\mathcal{M}} \sum_{m \in \mathcal{M}} \sum_{res} \frac{\Delta T_{res}(f, S_m)}{\sum_{f' \in \mathcal{F}_{base}} \Delta T_{res}(f', S_m)} \tag{3}$$

The expert prior weight $\mathcal{W}(f)$ of each function in $\mathcal{F}_{base}$ is assigned by experts based on domain knowledge about each function's role in anomaly detection and database context collection. First, functions that play critical roles in diagnosing representative anomalies are assigned higher weights. For instance, *LockAcquire()*, *LockRelease()*, and *ExecSeqScan()* are known to be highly influential in diagnosing lock contentions and scan inefficiencies. Second, functions necessary for collecting correlation identifiers that link resource usage to the corresponding database events are also assigned higher weights. For example, *mysql_execute_command()* is required to collect the correlations between SQL statements and query IDs. In contrast, other functions that are not on the core path of query execution or are less likely to cause performance bottlenecks receive lower weights. These assigned weights fall within the same range as $\mathcal{U}_{ano}$, i.e., $[0, |\mathcal{RS}|]$.

After that, all functions in $\mathcal{F}_{base}$ are ranked in descending order according to the diagnosis utility $\mathcal{U}(f)$. Then, we determine the largest $K$ such that the transaction throughput degradation caused by probing the top-$K$ functions is acceptable (e.g., $\leq 3\%$). We measure the throughput degradation through comprehensive stress testing across benchmark scenarios. The final selected top-$K$ functions form the optimal probe set $\mathcal{F}_{opt}$. We present a subset of the probe set for MySQL in [19].

### B. Online Anomaly-Aware Probing

To further improve the efficiency of metric collection (i.e., comprehensively cover anomaly-relevant functions with lower overhead), we design an online anomaly-aware probing mechanism that dynamically attaches the probe sets to database instances and adapts the frequency of message generation.

**Probe Set Repository.** The probe set is materialized as a standard-format JSON file. We maintain a probe set repository that maps probe set files to compatible DBMS versions, similar to an App store. Thus, a probe set can be built once and used everywhere. We can reuse probe sets for identical or compatible versions of the same DBMS, or even for different DBMSs derived from the same codebase. In addition to the optimal probe set, we also maintain some optional probe sets that can be attached on demand for DBMSs. We encourage

[1] Even for closed-source DBMSs, the vendor can usually provide the symbol table to customers along with the DBMS distribution package.

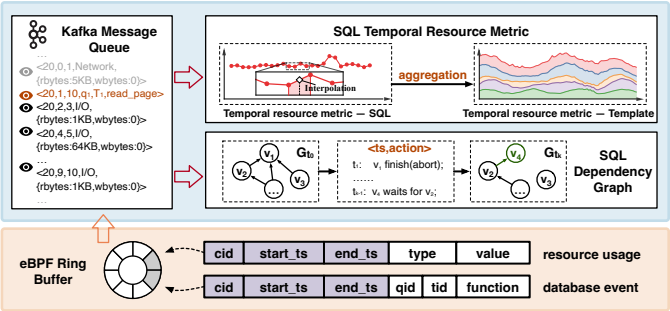



Fig. 3: Database context reconstruction.

DBMS providers to publish their own probe sets in the repository or work with us to build and publish probe sets.

**Probe Attachment.** We leverage the hot-pluggable nature of eBPF probes to support dynamic probe attach and detach without halting the database. This capability enables runtime balancing between probing overhead and probing coverage. For resource usage monitoring, we attach all required kprobes to relevant system calls. For database event monitoring, however, only the functions in the optimal probe set $\mathcal{F}_{opt}$ are instrumented with uprobes by default. During execution, the system periodically evaluates the runtime overhead of probing. If the observed overhead exceeds a predefined threshold, probes with the smallest utility $\mathcal{U}(f)$ are detached adaptively. Conversely, if a DBA requires finer-grained insights into specific metrics, the framework can dynamically add probes from the optional probe sets for those metrics.

**eBPF Message Format.** The metrics collected by probes are packed into eBPF messages. As shown at the bottom in Figure 3, both the resource usage messages and database event messages embed three fixed-length fields, including the correlation identifiers (*cid*), start timestamp (*start_ts*), and end timestamp (*end_ts*). These fields enable DBdoctor to associate resource usage messages with specific database event messages. For example, in MySQL, a SQL statement's disk I/O operation can be linked to the corresponding *vfs_read()* probe through the file descriptor [11], which uniquely identifies the accessed file object, and thread_id, which is related to the SQL statement. Combined with temporal alignment via timestamps, this design accurately attributes system-level I/O costs to individual SQL statements. In addition, resource usage messages include a *type* field that specifies the consumed resource dimension (e.g., I/O, CPU) and a *value* field that records the consumed resource volume. Database event messages include the query ID (*qid*), transaction ID (*tid*), function signature (*function*), and other essential database contexts.

**Adaptive Message Generation.** eBPF enables DBdoctor to collect fine-grained and lossless runtime metrics upon function invocation. However, naively generating one message for every probe invocation would result in excessive message volume and prohibitive runtime overhead, severely impacting normal query execution. To capture lossless anomaly-critical metric data while minimizing runtime overhead, DBdoctor embeds anomaly-aware logic directly within the eBPF programs to adaptively generate messages when necessary.

When collecting resource usage messages, for each resource dimension, we define a reporting interval and a spike threshold. During each probe invocation, the program first checks whether the observed resource consumption in this invocation is an anomalous behavior (e.g., resource usage exceeds the threshold). If an anomaly is detected, current resource usage is immediately constructed into an individual message and submitted to the ring buffer. Otherwise, resource usage aggregates with previously stored values in the eBPF Maps. Once the reporting interval elapses, the aggregated resource usage is packaged into a single message and submitted to the eBPF ring buffer. After submission, the user space is notified to consume messages for further analysis.

The generation of database event messages follows a similar strategy. Instead of generating a message for every function invocation, multiple homogeneous events within the same SQL execution are coalesced. For example, repeated *lock_acquire* events for a single SQL query are aggregated and pushed to user space only upon SQL completion or a transaction commit. However, if a lock wait time exceeds a predefined threshold, an immediate message is generated and pushed to the user space to highlight the anomaly. In practice, DBdoctor adopts the highest feasible aggregation interval (limited by eBPF constraints such as the stack size) to minimize runtime overhead, while preserving lossless anomaly observability.

**Message Consumption.** In the user space, a daemon continuously polls the eBPF ring buffer, retrieving messages and forwarding them into a Kafka message queue. The adoption of Kafka absorbs message bursts and prevents excessive consumption of system resources. The queued messages are then consumed by the *Database Context Reconstructor*, which performs the higher-level SQL context reconstruction (§V).

## V. Anomaly Diagnosis

The fine-grained event-based metrics enable precise and efficient anomaly diagnosis and resolution. However, it remains a significant challenge to identify the anomalies and their root causes from the massive raw messages. In this section, we present a solution that first reconstructs raw eBPF messages into two higher-level temporal database contexts: the *SQL-level temporal resource metric* (§V-A) and the *temporal SQL dependency graph* (§V-B). These contexts represent the real-time resource metrics across the query lifecycle and the inter-query dependencies. Then, the solution performs white-box anomaly diagnosis to identify anomalies and their root causes, enabling efficient and effective anomaly resolution (§V-C).

### A. SQL-Level Temporal Resource Metric

Since SQL statements with the same template are highly likely to exhibit similar resource consumption patterns and behavioral trends, we construct a temporal resource metric for each SQL template $\mathcal{Q}$ as Eq.4.

$$RM_{res}(\mathcal{Q}) = \{(t_i, v_{res}(t_i), \mathcal{A}_i) | t_i \in \mathcal{TS}\}, res \in \mathcal{RS} \quad (4)$$

where $\mathcal{RS}$ denotes the collection of monitored resource dimensions (e.g., CPU, I/O, lock count), $\mathcal{TS}$ denotes discrete time

points, and $v_{res}(t_i)$ represents the consumption of resource $res$ by template $\mathcal{Q}$ at $t_i$, and $\mathcal{A}_i$ represents the SQL actions that contribute to the resource consumption at $t_i$, which is the subset of predefined key action set $\mathcal{A}$ in the query lifecycle.

Notably, SQL templates are dynamically constructed online without requiring prior knowledge of the workload.

**Temporal Resource Metric Construction.** As shown in the upper part of Figure 3, DBdoctor performs a multi-stage aggregation process on the messages to construct the temporal resource metric for each SQL template.

(1) Resource-usage messages are mapped to database event messages via temporal and identifier correlations. A resource usage message $\langle cid, start_ts_0, end_ts_0, \cdots \rangle$ is mapped to a database event message $\langle cid, start_ts_1, end_ts_1, \cdots \rangle$ if they share the same $cid$ and the resource usage interval fails within the event's execution interval ($[start_ts_0, end_ts_0] \subseteq [start_ts_1, end_ts_1]$). This temporal constraint is required since different queries may reuse the same $cid$ over time.

(2) A temporal resource metric is constructed for each SQL statement by aggregating database event messages and their corresponding resource-usage messages. The cumulative resource consumption within each interval $[start_ts, end_ts]$ are converted into per-unit-time consumption rates $v_{res}(t_m) = \frac{consumption}{end_ts - start_ts}$, where $t_m$ represents the midpoint timestamp within the interval. Each consumption value is then assigned a SQL action label $act \in \mathcal{A}$ based on the function name in the corresponding database event message (e.g., function *ExecSeqScan()* is labeled as *scan*). The temporal resource metric for each SQL statement $q$ is thus defined as Eq.5.

$$RM_{res}(q) = \{(t_i, v_{res}(t_i), act_i) | t_i \in \mathcal{TS}\}, res \in \mathcal{RS} \quad (5)$$

(3) SQL temporal resource metrics are aggregated into template temporal resource metrics. For each template, we construct a unified temporal sequence $\mathcal{TS}(\mathcal{Q}) = \bigcup_{q \in \mathcal{Q}} \mathcal{TS}(q)$ by merging the timestamp sequences of all SQL statements. For each timestamp in this unified sequence, the per-unit resource consumption of the template is calculated by aggregating (e.g., sum, max, etc.) the resource usage of related SQLs. If a timestamp in the sequence falls within a query's execution interval but no consumption value is recorded for that query, the missing value is imputed using linear interpolation from neighboring points. Moreover, the action tags of SQL statements are aggregated into the template's action set.

### B. Temporal SQL Dependency Graph.

To provide observability of inter-query dependencies, we construct the temporal SQL dependency graph (TSDG) from lock-related database event messages. Let $G_t = (V_t, E_t)$ denote the dependency graph at time $t$. Each vertex $v \in V_t$ represents an active SQL, and each directed edge $(v_i \rightarrow v_j) \in E_t$ indicates that SQL $v_j$ is blocked, waiting for locks currently held by $v_i$. The directed edges $E_t$ are dynamically derived from lock-related events including *lock_acquire*, *lock_wait*, and *lock_release*. An edge $(v_i \rightarrow v_j)$ is inserted into $E_t$ when $v_i$ acquires a lock and $v_j$ subsequently issues a *lock_wait* event on the same lock. The edge is removed once $v_i$ releases the lock, as indicated by a *lock_release* event.

We also correlate SQLs to transactions using the transaction ID in messages, and group the SQLs of the same transaction in the TSDG. Thus, TSDG can reflect lock dependencies not only among SQLs but also among transactions. Moreover, in some DBMSs (e.g., MySQL), the kernel may add additional locks for transactions. Such lock events not triggered by SQL are attached to transactions in TSDG.

As shown in the upper part of Figure 3, to reduce storage cost while ensuring graph reconstruction efficiency, we adopt a hybrid snapshot-and-delta storage strategy for TSDG. Rather than continuously persisting the full graph state, we periodically store a complete snapshot of the dependency graph $G_{snapshot_i}$. The intermediate changes between snapshots are captured as incremental updates $\Delta G_t = (\Delta V_t, \Delta E_t)$, where $\Delta V_t$ and $\Delta E_t$ denote newly added or removed nodes and edges relative to $G_{t-1}$. To reconstruct the dependency graph $G_t$ at any time point $t$, we first retrieve the most recent preceding snapshot $G_{snapshot_i}$ where $snapshot_i \leq t$ and then apply all delta updates between $snapshot_i$ and $t$. This snapshot-delta sequence forms the complete TSDG structure over a time horizon $\mathcal{TS}$: $\{G_{snapshot_i}, \Delta G_{t_i}, \Delta G_{t_{i+1}}, ...\}$.

### C. Anomaly Identification and Resolution

Based on the temporal database contexts, we introduce how DBdoctor identifies anomalies (§V-C1) and their root causes (§V-C2), and performs anomaly resolution (§V-C3).

*1) Anomaly Identification:* DBdoctor identifies anomalies under two conditions: (1) system-level anomaly triggered when the consumption of a resource (e.g., CPU, memory, or I/O bandwidth) or the overall system throughput degradation exceeds a predefined threshold; and (2) SQL-level anomaly triggered when a query's temporal resource metric or latency significantly deviates from its historical behavior. Such SQL-level deviations are captured using time-series analysis techniques, including statistical outlier detection [21], [33] and change-point detection [23], [34]. Upon satisfying either condition, DBdoctor initiates the diagnosis pipeline.

*2) Root Cause Identification.:* To achieve accurate root cause identification, DBdoctor follows a two-step procedure. We first identify the root-cause SQLs (RC-SQLs) of the anomaly, leveraging the SQL temporal resource metrics and dependency graph. Then, we feed both internal and external database metrics into an inference engine to identify the accurate root causes of the anomaly.

**RC-SQL Identification.** Since multiple anomalies may coexist, DBdoctor performs independent RC-SQL identification for each anomaly. For an anomaly detected within the interval $[t_s, t_e]$, DBdoctor retrieves the SQL dependency graph and the SQL temporal resource metrics over an expanded interval $[t_s - \delta, t_e]$, as RC-SQLs triggering the anomaly may start before the SQL anomaly becomes observable.

For lock-related anomalies, such as long lock waits or deadlocks, DBdoctor identifies RC-SQLs through structural analysis of the TSDG. For deadlocks, the DBMS kernel

generally rolls back at least one transaction in the deadlock cycle. Such rollback events are captured and reflected on the TSDG. SQL with deadlock rollbacks are identified as the RC-SQLs. For long lock waits, a configurable timeout (e.g., 1 s) is used to flag anomalies. Then, DBdoctor traverses the TSDG from the waiting SQL to locate the RC-SQLs that hold the blocking locks. If no such SQL exists in the TSDG, the transaction itself is identified as the root cause.

For resource-related anomalies, DBdoctor identifies RC-SQLs through correlation analysis of the system-level temporal metrics and the SQL template temporal metrics of the anomalous resources. The SQL statements belonging to the template with the highest Spearman's Rank correlation coefficient [36] are designated as RC-SQLs, as their execution most closely aligns with the anomalous resource fluctuations.

**Root Cause Inference.** After RC-SQL identification, DBdoctor infers the root cause through an inference engine. It first constructs a multi-dimensional *contextual feature vector* $F_C$, which integrates three categories of information: (1) SQL action features, including the temporal resource metrics $RM_{res}(\mathcal{Q})$ of RC-SQLs during the anomalous interval, along with the embedded action tags that provide database contexts; (2) SQL structure features, including SQL statements and their execution plans. (3) Database metrics, such as database settings and database-level statistics. Given $F_C$, the inference engine evaluates a hypothesis set $\mathcal{H} = \{h_1, h_2, ..., h_k\}$, where each $h_i$ represents a known database root cause type, such as *Missing Index*, *Deadlocks*, and computes the causal score $S(h_i|F_C)$ for each hypothesis $h_i$. Specifically, each hypothesis $h_i$ is associated with a set of diagnostic rules derived from domain knowledge and historical anomaly data. Each rule specifies a pattern over $F_C$ (e.g., sustained lock-wait spikes on RC-SQLs) and contributes a weighted evidence score when the pattern is matched. The score $S(h_i \mid F_C)$ is computed by aggregating the evidence scores contributed by all matched rules for hypothesis $h_i$. Hypotheses whose score $S(h_i)$ exceeds a predefined threshold are designated as root causes.

It is worth noting that the fine-grained, context-aware metrics collected by DBdoctor can be seamlessly integrated with other diagnosis approaches, such as machine learning–based or LLM–driven methods, for better accuracy and interpretability.

*3) Anomaly Resolution:* DBdoctor employs the following performance anomaly resolution methods to ensure both rapid recovery and sustained system stability.

**Instant Resolution.** DBdoctor immediately mitigates performance degradation through workload control when RC-SQLs are identified. Specifically, it either terminates sessions for RC-SQLs to promptly reduce resource contention or applies SQL throttling to limit the concurrency of RC-SQLs.

**Long-Term Resolution.** To prevent recurrent anomalies, DBdoctor employs targeted optimization strategies, such as SQL rewriting [28], parameter tuning [38], and index recommendation [40], based on the identified root causes. We combine rule-based resolution and LLM-based resolution in DBdoctor. We also sandbox the database optimizer and key execution logic to quickly audit candidate resolutions (e.g., adding an index) before executing them, confirming their effectiveness (e.g., whether the optimizer selects the added index). Furthermore, we build a knowledge base of anomalies and their corresponding effective solutions, enabling us to use RAG to improve the accuracy of LLM-based resolution. Due to space limitations, we do not discuss the details of resolution.

## VI. Industrial Deployment

In this section, we describe the industrial deployment of DBdoctor. We first introduce how DBdoctor ensures security in productions and then explain how the probe set is constructed and maintained for evolving DBMSs. Moreover, we present DBdoctor' diagnosis workflow through a case study.

### A. Security Discussion

Although deploying eBPF in production systems may raise security concerns, DBdoctor uses eBPF in a strictly observational and security-preserving manner. All eBPF programs are statically verified by the kernel-level eBPF verifier [9] before execution, thereby preventing unsafe behaviors such as infinite loops or illegal memory accesses, ensuring kernel security. Moreover, DBdoctor restricts eBPF usage to read-only tracing mechanisms (e.g., kprobes and uprobes) that observe execution events without modifying kernel or DBMS behavior, ensuring that normal database execution semantics remain unaffected. Finally, DBdoctor collects only execution metadata rather than user data, and all collected information is processed locally within the administrative domain, preserving data security.

### B. Probe Set Construction and Maintenance

We next describe the probe set management for evolving DBMSs. Although probe set construction requires domain expertise and symbol-table analysis, this does not limit the generalizability of DBdoctor. First, expert involvement is a one-time, offline effort per DBMS engine; once constructed, the probe set can be reused across diverse workloads without further manual tuning. Second, probe construction methodology is largely transferable across DBMSs, as modern engines share similar query and transaction execution pipelines and usually exhibit anomalies at functionally equivalent execution stages. Consequently, extending DBdoctor to a new DBMS primarily involves mapping predefined logical database events to concrete functions rather than exhaustive symbol-table analysis. For DBMSs derived from mainstream engines, probe sets can often be inherited with minimal adaptation. As shown in Table I, this design allows DBdoctor to scale to new systems and versions with modest engineering effort. Moreover, DBdoctor applies a lightweight collaboration model with the vendors or communities for both closed- and open-source DBMSs. Based on historical probe sets, we define a set of logical probe specifications describing which functional events to monitor and what runtime information to capture. DBMS vendors or communities only provide symbol mappings and the names of relevant variables. This significantly improves the operational efficiency and usability of DBMSs, thereby motivating vendors and communities to cooperate. We have

TABLE I: Manual effort for the probe set construction.

| Database Type | Manual Effort (Person-Month) |
|---|---|
| Complete new DBMS | (6 – 7) × 2 |
| Variant of a probed DBMS | 1 – 2 |
| DBMS New version (Major change) | 0.5 – 1 |
| DBMS New version (Minor change) | 0.1 – 0.3 |

collaborated with over 10 vendors and communities, including MySQL, OceanBase, and Vastbase [20], under this model.

### C. Case Study

To illustrate the diagnosis workflow of DBdoctor, we use the multi-root-cause anomaly described in §II-A as an example. After detecting the anomaly, DBdoctor performs both RC-SQL identification and root cause inference. By traversing the global dependency graph within the anomalous interval and performing a correlation analysis between the resource metrics of active SQL templates and the system-level metrics, DBdoctor identifies `DELETE FROM Table_A WHERE field_A < ? LIMIT ?` (blocking concurrent queries) and `UPDATE Table_B WHERE field_B = ?` (high correlation with the anomalous resource) as RC-SQLs. Based on the RC-SQLs' action tags (*scan* and *update_index*) within the anomalous interval, along with SQL and database features, the subsequent analysis attributes the root causes to *Missing Index* and *Redundant Index*. Then DBdoctor recommends appropriate optimizations, including creating and deleting indexes.

## VII. Evaluation

In this section, we compare DBdoctor to state-of-the-art diagnosis solutions through following experiments: (1) DBdoctor's anomaly diagnosis accuracy, including identification of root causes and the associated root-cause SQLs (§VII-B), (2) DBdoctor's monitoring overhead compared to existing monitoring tools (§VII-C), (3) the ablation study (§VII-D), (4) the generality of DBdoctor (§VII-E), and (5) evaluation results in production environments.

### A. Experimental Setup

Both DBdoctor and databases are deployed on a server running Linux v4.18 with 8 CPU cores (16 threads), 32 GB DRAM, and 500 GB SSD storage. We utilize the official bpftool [4] to manage the eBPF programs.

*1) Default configuration:* We conduct experiments using two popular databases: MySQL 8.0.22 [13] and PostgreSQL 15.2 [25]. By default, we set the isolation level to serializable.

*2) Baselines:* In our experiments, we compare DBdoctor against the following baselines. For root cause SQL (RC-SQL) identification, we compare DBdoctor against **PinSQL** [29], a state-of-the-art solution that identifies RC-SQLs using the active session metric (i.e., the number of active sessions related to the query at a given timestamp). For root cause diagnosis, we evaluate DBdoctor against the machine learning-based approach **DBPA** [26] (including OC-SVM [31] and SVDD [32]) and the LLM-driven approach **D-Bot** [41] (deepseek-v3 without fine-tuning). All baseline approaches rely on sample-based monitoring tools, including an open-source tool Dool [7] to collect system-level metrics and integrated database plugins, such as pg_stat_activity [12], to collect query-level statistics.

*3) Benchmarks:* We conducted experiments on public benchmarks and datasets collected in production environments.
**(1) TPC-C**. We use 100 warehouses by default. Following previous works [39], [45], we exclude *think time*.
**(2) TPC-H**. We use a scale factor of 10 (10GB dataset). We conduct experiments on all 22 TPC-H queries.
**(3) Sysbench**. We provisioned 16 sbtest tables, each containing 1 million rows, for a total dataset of approximately 3.2 GB.
**(4) Real-world.** It is the real-world workload over a period of time in a production environment, comprising 203 anomaly cases due to a single root cause and 315 cases due to multiple root causes. As shown in Table II, we list nine representative root cause categories in this workload, along with their distributions. We use this workload to evaluate the diagnosis accuracy in §VII-B, §VII-D, and §VII-E.

*4) Evaluation Metrics:* Following prior work [29], we evaluate RC-SQL identification using *HitRate* and Mean Reciprocal Rank (*MRR*). For each anomaly case within the evaluation, the per-case *HitRate* can be calculated by $\frac{|P_i \cap G_i|}{|G_i|}$, where $G_i$ represents the ground-truth RC-SQL set, and $P_i$ represents the predicted set. The overall *HitRate* is the average across all cases. *MRR* measures ranking quality, calculated by $MRR = \frac{1}{|\mathcal{Q}|}\sum_{i=1}^{|\mathcal{Q}|}\frac{1}{rank_i}$, where $rank_i$ represents the position of the first correctly predicted RC-SQL within the ground-truth set. For root cause identification, we report *Precision*, *Recall*, and *F1-score*, computed in a multi-label setting.

### B. Anomaly Diagnosis Accuracy

In this experiment, we evaluate DBdoctor's effectiveness for anomaly diagnosis using PostgreSQL. We evaluate the identification accuracy of RC-SQLs in §VII-B1 and evaluate the identification accuracy of root causes in §VII-B2.

*1) Root cause SQL identification:* We compare DBdoctor's RC-SQL diagnosis accuracy with PinSQL. As shown in Table III, DBdoctor consistently outperforms PinSQL. In single-cause scenarios, DBdoctor improves *HitRate* and *MRR* by 57.1 and 48.6 percentage points, respectively. The advantage is more obvious for multi-cause anomalies, where DBdoctor improves *HitRate* and *MRR* by 59.2 and 59.1 percentage points, respectively.

These performance improvements stem from two key factors. First, DBdoctor uses SQL-level temporal resource metrics and dependency graphs to directly correlate anomalous system behavior with per-template SQL resource usage and dependencies, enabling precise RC-SQL identification. In contrast, PinSQL identifies RC-SQLs indirectly by analyzing the number of active sessions per SQL template to infer which queries are affected by RC-SQLs, then clustering SQL templates by business logic and analyzing their historical behavior to infer the RC-SQLs. This approach may lead to inaccuracies when affected queries are immediately aborted or when dependencies are complex. Second, DBdoctor captures fine-grained anomalous events, enabling precise detection of short-lived performance spikes. In contrast, PinSQL relies

TABLE II: Representative root causes in the real-world workload.

| Root Cause | Description | Distribution (% anomalies) |
|---|---|---|
| Long transactions | Long-running transactions caused by inefficient business logic. | 27.57% |
| Uncommitted transactions | Transactions suspended without commit or rollback (such as waiting for third-party API responses). | 16.43% |
| Missing indexes | Lack of indexes on large tables result in slow full table scans. | 6.67% |
| Redundant indexes | Unused indexes result in increased update overhead. | 5.71% |
| Lock contention | Intensive concurrent access on the same item, leading to long lock waits. | 19.43% |
| Deadlocks | Cyclic lock dependencies between transactions can cause the database to become unresponsive. | 17.52% |
| Excessive data scan | Wide query ranges or data skew can lead to large scan volumes. | 28.57% |
| Misconfigured parameters | Inappropriate parameters can lead to poor performance. | 7.62% |
| Poorly written SQL | Poorly written SQLs, such as `SELECT *, poor joins`, lead to bad performance. | 45.71% |

TABLE III: Accuracy of RC-SQL diagnosis on Real-world.

| Method | Single Cause Anomaly | | Multi-Cause Anomaly | |
|---|---|---|---|---|
| | HitRate | MRR | HitRate | MRR |
| PinSQL | 0.343 | 0.458 | 0.271 | 0.313 |
| DBdoctor | 0.914 | 0.944 | 0.863 | 0.904 |

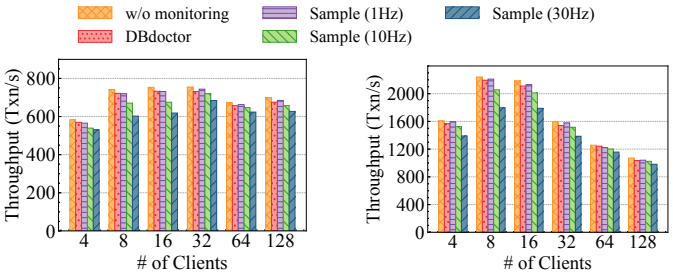
TABLE IV: Accuracy of root cause diagnosis on Real-world.

| Method | Single Cause Anomaly | | | Multi-Cause Anomaly | | |
|---|---|---|---|---|---|---|
| | Precision | Recall | F1-score | Precision | Recall | F1-score |
| OC-SVM (1 Hz) | 0.688 | 1 | 0.803 | 0.268 | 0.193 | 0.225 |
| OC-SVM (10 Hz) | 0.713 | 1 | 0.832 | 0.282 | 0.222 | 0.249 |
| OC-SVM (30 Hz) | 0.734 | 1 | 0.846 | 0.311 | 0.270 | 0.289 |
| SVDD (1 Hz) | 0.683 | 1 | 0.799 | 0.421 | 0.372 | 0.395 |
| SVDD (10 Hz) | 0.720 | 1 | 0.837 | 0.464 | 0.423 | 0.442 |
| SVDD (30 Hz) | 0.742 | 1 | 0.852 | 0.481 | 0.452 | 0.466 |
| D-Bot (1 Hz) | 0.717 | 1 | 0.879 | 0.478 | 0.402 | 0.437 |
| D-Bot (10 Hz) | 0.753 | 1 | 0.859 | 0.519 | 0.443 | 0.479 |
| D-Bot (30 Hz) | 0.775 | 1 | 0.873 | 0.537 | 0.474 | 0.503 |
| DBdoctor | 0.943 | 1 | 0.971 | 0.938 | 0.911 | 0.924 |

on coarse-grained (e.g., 1s or 1min) metrics that may miss transient anomalies.

*2) Root cause identification:* We then evaluate the accuracy of identifying root causes. We compare DBdoctor with three diagnosis baselines (OC-SVM, SVDD, and D-Bot) with various sampling frequencies (1 Hz, 10 Hz, and 30 Hz). As shown in Table IV, although high sampling frequency can achieve higher diagnosis accuracy for baseline approaches compared to low sampling frequency, DBdoctor outperforms all baselines in all scenarios. For single-cause anomalies, DBdoctor achieves a precision of 0.943, perfect recall, and an F1 score of 0.971, exceeding the best-performing baseline (D-Bot 1 Hz) by 9.2 percentage points. For multi-cause anomalies, DBdoctor remains highly accurate while the identification accuracy of baseline approaches degrades significantly. Specifically, DBdoctor attains an F1-score of 0.924, exceeding the best-performing baseline (D-Bot 30 Hz) by 42.1 percentage points. These improvements are primarily due to the white-box diagnosis capability of DBdoctor.

In single-root-cause scenarios, baseline approaches perform poorly on anomalies caused by *long transactions*, *uncommitted transactions*, and *redundant indexes*. The best baseline (D-Bot 1 Hz) achieves only 0.810, 0.795, and 0.5 F1-score, respectively. The key limitation is that existing approaches rely on aggregated metrics without directly linking anomalous phenomena to SQL execution. For example, redundant indexes introduce write amplification and high CPU and I/O usage due to index maintenance. Existing monitoring tools expose only aggregated metrics, such as numerous *WALWriteWaitEvent* and *DataFileWrite/ReadEvent*. However, similar metric patterns also arise from other causes, such as long-running transactions that incur extensive data updates. As a result, diagnosis methods often misattribute these root causes. In contrast, DBdoctor correlates fine-grained resource consumption with SQL actions over time, accurately identifying redundant indexes by revealing excessive index update overhead.




(a) MySQL (b) PostgreSQL

Fig. 4: Impact of concurrency on TPC-C.

The advantage of DBdoctor is more pronounced in multi-cause scenarios, where multiple anomalies co-occur and interact. For example, an uncommitted transaction and an update without an index jointly cause contention. Baseline methods can only detect lock contention, but they fail to identify uncommitted transactions and missing index due to the lack of fine-grained internal database observability.

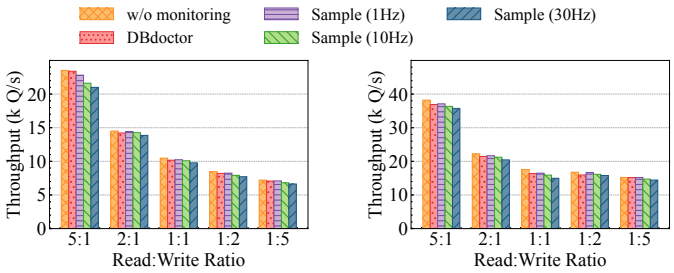
**Summary.** DBdoctor provides fine-grained metrics and performs white-box diagnosis, thus achieving high accuracy on both RC-SQL and root cause identification.

### C. Diagnosis Overhead

In this experiment, we evaluate the overhead of DBdoctor against the sample-based monitoring used in baselines with varying sampling frequencies.

**Under varying concurrency.** We use TPC-C benchmark to evaluate the impact of concurrency by varying the number of clients. As shown in Figure 4, compared to normal execution, the throughput of DBdoctor declines 3.9% and 3.5% on PostgreSQL and MySQL, respectively. The performance overhead is comparable to sample-based collection at 1 Hz (with a gap of less than 1%), while incurring lower overhead than sample-based collection at 10 Hz and 30 Hz.

**Under varying write ratio.** As shown in Figure 5, we use Sysbench to evaluate the performance overhead with different write ratios. The average throughput of DBdoctor decreases by 2.1% on MySQL and 3.8% on PostgreSQL versus normal execution. It achieves performance comparable to the sample-based collection strategy at 1 Hz, while incurring lower overhead than strategies at 10 Hz and 30 Hz.




(a) MySQL (b) PostgreSQL

Fig. 5: Impact of write ratio on Sysbench.

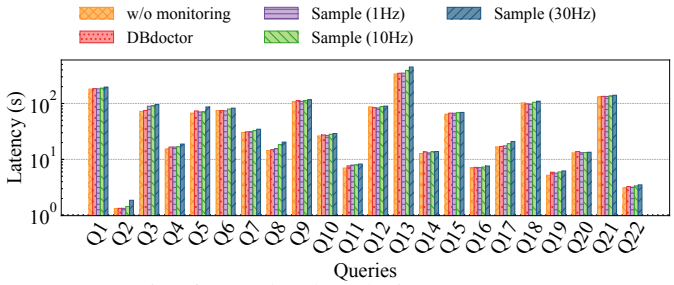



Fig. 6: Overhead analysis on TPC-H.

**Overhead on TPC-H.** We evaluate the performance overhead on analytical workloads with all queries of TPC-H. As shown in Figure 6, DBdoctor incurs an average overhead of only 3.9% on MySQL and 3.8% on PostgreSQL, which is comparable to the sample-based collection strategy at 1 Hz and lower than the overhead of strategies at 10 Hz and 30 Hz.

Furthermore, as shown in Table V, we list the memory and CPU usage of DBdoctor using TPC-C in MySQL. DBdoctor introduces the acceptable performance overhead. Specifically, in terms of CPU utilization, DBdoctor incurs extra CPU utilization of approximately 4.91 to 9.02 percentage points. For memory overhead, DBdoctor exhibits even smaller impact. Under low concurrency scenarios (e.g., 4 clients), memory consumption increases by only about 3.61 MB. Even at peak load with 32 clients, the total memory overhead is 144.01 MB, a relative increase of under 15.4%.

**Summary.** DBdoctor incurs negligible performance overhead, comparable to or less than existing sample-based monitoring approaches, making it suitable for deployment in production.

### D. Ablation Study

To understand the contribution of key design components in DBdoctor, we conduct an ablation study on three major aspects: (1) adaptive message generation, (2) probe set coverage, and (3) SQL dependency graph construction. As shown in Table VI, reducing the probe set coverage (DBdoctor (- Probe Set)) incurs lower overhead but leads to a drop in diagnosis accuracy. In contrast, adding more probes (DBdoctor (+ Probe Set)) increases overhead while bringing only marginal accuracy improvement. This indicates that DBdoctor can select the optimal probe set that achieves high diagnosis accuracy with low overhead. Moreover, we study the effect of the adaptive message generation mechanism. Completely disabling aggregation and generating one message per probe invocation leads to an excessive number of messages and prohibitive performance degradation, causing the DBMS to remain persistently overloaded and thus making diagnosis impractical. Therefore, we adopt a relaxed ablation by shortening the aggregation interval to one quarter of the default setting in this experiment. As shown in Table VI, this weakened strategy (DBdoctor (w/o Adaptive Message Generation)) increases runtime overhead substantially, with QPS degradation rising from 4.81% to 17.53%, while the diagnosis accuracy remains almost unchanged. This demonstrates that adaptive message generation is critical for controlling overhead without sacrificing diagnostic effectiveness. Finally, disabling SQL dependency graph construction slightly reduces overhead but causes a noticeable drop in diagnosis accuracy, confirming its importance for root-cause identification.

**Summary.** The ablation study shows that the optimal probe set, adaptive message generation, and SQL dependency graph are all essential for achieving high diagnosis accuracy with low runtime overhead.

TABLE V: Memory and CPU usage on TPC-C.

| # of clients | | 4 | 8 | 16 | 32 | 64 | 128 |
|---|---|---|---|---|---|---|---|
| Avg CPU (%) | w/o monitoring | 23.57 | 38.23 | 40.23 | 35.79 | 36.18 | 41.31 |
| | w DBdoctor | 30.78 | 47.25 | 49.12 | 41.55 | 41.09 | 47.81 |
| Avg Mem (MB) | w/o monitoring | 415.83 | 459.41 | 476.96 | 932.52 | 1189.85 | 1465.89 |
| | w DBdoctor | 419.44 | 494.91 | 503.25 | 1076.53 | 1222.81 | 1462.62 |

TABLE VI: Ablation study of DBdoctor on Real-world.

| Method | Accuracy | | | QPS degradation(%) |
|---|---|---|---|---|
| | Precision | Recall | F1-score | |
| DBdoctor | 0.941 | 0.950 | 0.946 | 4.81 |
| DBdoctor (- Probe Set) | 0.854 | 0.895 | 0.874 | 3.22 |
| DBdoctor (+ Probe Set) | 0.944 | 0.954 | 0.944 | 7.49 |
| DBdoctor (w/o Adaptive Message Generation) | 0.940 | 0.949 | 0.945 | 17.53 |
| DBdoctor (w/o Dependency Graph) | 0.868 | 0.901 | 0.884 | 3.96 |

### E. Evaluation on Generality

We evaluate the generality of DBdoctor from two perspectives: whether the framework can be deployed across various DBMSs, and whether its monitoring framework can be used by existing diagnosis methods. As shown in Table VII, we evaluate DBdoctor on MySQL, PostgreSQL, and a distributed database OceanBase [37]. Across all three DBMSs, DBdoctor achieves over 93% diagnosis accuracy while keeping the runtime overhead below 5%, demonstrating that it generalizes well across heterogeneous DBMSs. We further assess whether DBdoctor 's monitoring framework can benefit existing diagnosis methods. As shown in Table VIII, we integrate DBdoctor's monitoring framework into the LLM-based diagnosis approach D-Bot and conduct experiments on PostgreSQL with the real-world workload. Replacing D-Bot's original metrics with those collected from DBdoctor significantly improves its diagnosis accuracy. The precision increases by 16.2 percentage points for single-cause anomalies and by 32.8 percentage points for multi-cause anomalies.

**Summary.** DBdoctor achieves good generalizability across various DBMSs and diagnosis methods.

TABLE VII: Generalizability of DBdoctor on various DBMSs.

| Database | Accuracy | | | QPS degradation (%) |
|---|---|---|---|---|
| | Precision | Recall | F1-score | |
| DBdoctor (PostgreSQL) | 0.941 | 0.950 | 0.946 | 4.81 |
| DBdoctor (MySQL) | 0.944 | 0.951 | 0.948 | 2.91 |
| DBdoctor (OceanBase) | 0.931 | 0.945 | 0.939 | 3.09 |

TABLE VIII: Generalizability of the monitoring framework.

| Method | Single Cause Anomaly | | | Multi-Cause Anomaly | | |
|---|---|---|---|---|---|---|
| | Precision | Recall | F1-score | Precision | Recall | F1-score |
| D-Bot | 0.717 | 1 | 0.879 | 0.478 | 0.402 | 0.437 |
| D-Bot w DBdoctor | 0.879 | 1 | 0.932 | 0.806 | 0.614 | 0.697 |

### *F. Evaluation in Production Environments*

DBdoctor has been deployed in the production environment comprising thousands of database instances at Hisense, a major global smart TV manufacturer and online video service provider with over 100 million active users. Smart TV functions such as membership, advertising, shopping, and education all require database access. According to assessments by DBAs, DBdoctor achieves over 90% diagnosis accuracy on daily workloads. The runtime overhead remains below 5%, which is acceptable for production. Through continuous diagnosing, DBdoctor helps maintain stable database performance during workload surges, such as live broadcasts of the FIFA World Cup and the CMG Spring Festival Gala.

In addition, DBdoctor has obtained the Advanced (highest) level certification for SQL Quality Management Platform Capability [19], issued by the China Academy of Information and Communications Technology (CAICT) [5], the most widely recognized database product certification organization in China. This provides independent third-party verification of the industrial applicability of DBdoctor.

## VIII. Related Work

**Database performance diagnosis.** This is the process of identifying the root causes behind performance anomalies. Automated diagnosing approaches [29], [30], [35], [41], [44], are proposed to improve the diagnosis accuracy and reduce human effort. For example, iSQUAD [30] leverages machine learning to diagnose the root causes of intermittent slow queries in cloud databases. D-Bot [41] uses LLM to automatically acquire knowledge from documentation and generate diagnostic reports with solutions. PinSQL [29] identifies root cause SQLs that are truly responsible for the performance anomalies using session metrics and application logics. However, these approaches rely on existing sample-based external monitoring tools, which fail to capture fine-grained metrics or provide database internal context observability necessary for accurate diagnosis. DBdoctor proposes an event-driven metric collection framework to capture fine-grained database internal metrics. Moreover, we model the metric data into SQL temporal resource metrics and dependency graphs to enable white-box anomaly diagnosis. Notably, our metric collection and modeling methods can complement existing diagnostic approaches to improve diagnosis accuracy.

**eBPF for databases.** The eBPF technology has been widely used in many applications, including network load balancing and observability [6], runtime security monitoring [10], general-purpose system observability [3], and distributed protocols [42]. A few studies have introduced eBPF into database systems. For example, Dint [43] leverages eBPF to accelerate the performance of distributed transactions. In contrast to those approaches, DBdoctor focuses on leveraging eBPF to monitor database events and context, aggregating these events into SQL-level metrics for database performance diagnosis.

## IX. Discussion

DBdoctor uses fine-grained event-based metric collection and modeling to comprehensively capture anomaly events and reconstruct database contexts for anomaly diagnosis and resolution with low overhead. Through successful and wide commercial deployments, DBdoctor proves this to be a viable approach that fundamentally improves the efficiency and accuracy of online performance anomaly diagnosis for databases. And the framework of DBdoctor is extensible for applying new diagnosis and resolution methods.

Identifying the eBPF probe sets requires the participation of DBAs and kernel experts. This is a one-time job for a DBMS, and DBMS providers are cooperating with us to build probe sets, because customers are willing to choose DBMSs with better diagnostic tools. Moreover, the benefits extend beyond database anomaly diagnosis. Our framework, in fact, builds an ecosystem that solidifies knowledge of the database kernel and accumulates critical runtime data. The probe set repository, event contexts, and verified effective root causes and resolutions provide substantial, valuable data to address the major pain point of insufficient data for AI4DB research.

## X. Conclusion

In this paper, we present DBdoctor, a non-intrusive performance-diagnosis platform applicable to various database systems. DBdoctor proposes an adaptive, event-based metric collection framework that captures fine-grained internal and external database metrics with low overhead. The collected metrics are modeled as SQL temporal resource metrics and dependency graphs, providing internal observability of query executions and enabling white-box anomaly diagnosis. Extensive evaluations on both popular benchmarks and real-world workloads demonstrate that DBdoctor outperforms other diagnosis approaches in root-cause identification accuracy while incurring comparable or lower overhead.

## Acknowledge

The work is supported by the National Natural Science Foundation of China (Grant Nos. 62441230, 62472429, U23A20299, and 62502521), the Outstanding Innovative Talents Cultivation Funded Programs 2025 of Renmin University of China, the Fundamental Research Funds for the Central Universities, and the Research Funds of Renmin University of China (Grant NO. 24XNKJ10), the A3 Foresight Program No. 62461146205, and Beijing Natural Science Foundation (Grant No. 4254083).

## REFERENCES


[1] Akamai state of online retail performance spring 2017. https://www.scribd.com/document/451425146/akamai-state-of-online-retail-performance-spring-2017, 2025.

[2] Amazon study: Every 100ms in added page load time cost 1% in revenue. https://www.conductor.com/academy/page-speed-resources/faq/amazon-page-speed-study/, 2025.

[3] Bcc - tools for bpf-based linux io analysis, networking, monitoring, and more. https://github.com/iovisor/bcc, 2025.

[4] bpftool. https://github.com/libbpf/bpftool.git, 2025.

[5] China academy of information and communications technology. https://www.caict.ac.cn/english/, 2025.

[6] Cilium - cloud native, ebpf-based networking, observability, security. https://cilium.io/, 2025.

[7] Dool, linux cli tool providing real-time system resource monitoring. https://github.com/scottchiefbaker/dool.git, 2025.

[8] ebpf, dynamically program the kernel for efficient networking, observability, tracing, and security. https://ebpf.io/, 2025.

[9] ebpf verifier. https://docs.kernel.org/bpf/verifier.html, 2025.

[10] Falco - detect security threats in real time. https://falco.org/, 2025.

[11] File descriptor. https://en.wikipedia.org/wiki/File_descriptor, 2025.

[12] Monitoring database activity – postgresql. https://www.postgresql.org/docs/current/monitoring-stats.html, 2025.

[13] mysql-server. https://github.com/mysql/mysql-server.git, 2025.

[14] perf: Linux profiling with performance counters. https://perfwiki.github.io/main/, 2025.

[15] Performance monitoring and tuning tools - sql server. https://learn.microsoft.com/en-us/sql/relational-databases/performance/performance-monitoring-and-tuning-tools?view=sql-server-ver17, 2025.

[16] pg_stat_statements, track statistics of sql planning and execution. https://www.postgresql.org/docs/current/pgstatstatements.html, 2025.

[17] Prometheus - monitoring system & time series database. https://prometheus.io/, 2025.

[18] Sql trace - oracle. https://docs.oracle.com/cd/E18727_01/doc.121/e13471/T2650T455086.htm, 2025.

[19] Supplementary material. https://github.com/neighthorn/DBdoctor.git, 2025.

[20] Vastbase - real-time data integration platform. https://docs.tapdata.io/connectors/on-prem-databases/vastbase/, 2025.

[21] Markus M. Breunig, Hans-Peter Kriegel, Raymond T. Ng, and Jörg Sander. LOF: identifying density-based local outliers. In SIGMOD Conference, pages 93–104. ACM, 2000.

[22] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. Benchmarking cloud serving systems with YCSB. In Joseph M. Hellerstein, Surajit Chaudhuri, and Mendel Rosenblum, editors, Proceedings of the 1st ACM Symposium on Cloud Computing, SoCC 2010, Indianapolis, Indiana, USA, June 10-11, 2010, pages 143–154. ACM, 2010.

[23] Joshua Corneck, Edward A. K. Cohen, James S. Martin, and Francesco Sanna Passino. Online bayesian changepoint detection for network poisson processes with community structure. Stat. Comput., 35(3):75, 2025.

[24] Karl Dias, Mark Ramacher, Uri Shaft, Venkateshwaran Venkataramani, and Graham Wood. Automatic performance diagnosis and tuning in oracle. In Second Biennial Conference on Innovative Data Systems Research, CIDR 2005, Asilomar, CA, USA, January 4-7, 2005, Online Proceedings, pages 84–94. www.cidrdb.org, 2005.

[25] Jose M. Faleiro and Daniel J. Abadi. Rethinking serializable multiversion concurrency control. Proc. VLDB Endow., 8(11):1190–1201, 2015.

[26] Shiyue Huang, Ziwei Wang, Xinyi Zhang, Yaofeng Tu, Zhongliang Li, and Bin Cui. DBPA: A benchmark for transactional database performance anomalies. Proc. ACM Manag. Data, 1(1):72:1–72:26, 2023.

[27] Jay Kreps, Neha Narkhede, Jun Rao, et al. Kafka: A distributed messaging system for log processing. In Proceedings of the NetDB, volume 11, pages 1–7. Athens, Greece, 2011.

[28] Zhaodonghui Li, Haitao Yuan, Huiming Wang, Gao Cong, and Lidong Bing. LLM-R2: A large language model enhanced rule-based rewrite system for boosting query efficiency. Proc. VLDB Endow., 18(1):53–65, 2024.

[29] Xiaoze Liu, Zheng Yin, Chao Zhao, Congcong Ge, Lu Chen, Yunjun Gao, Dimeng Li, Ziting Wang, Gaozhong Liang, Jian Tan, and Feifei Li. Pinsql: Pinpoint root cause sqls to resolve performance issues in cloud databases. In ICDE, pages 2549–2561. IEEE, 2022.

[30] Minghua Ma, Zheng Yin, Shenglin Zhang, Sheng Wang, Christopher Zheng, Xinhao Jiang, Hanwen Hu, Cheng Luo, Yilin Li, Nengjun Qiu, Feifei Li, Changcheng Chen, and Dan Pei. Diagnosing root causes of intermittent slow queries in large-scale cloud databases. Proc. VLDB Endow., 13(8):1176–1189, 2020.

[31] Carla Sauvanaud, Mohamed Kaâniche, Karama Kanoun, Kahina Lazri, and Guthemberg Silvestre. Anomaly detection and diagnosis for cloud services: Practical experiments and lessons learned. J. Syst. Softw., 139:84–106, 2018.

[32] Carla Sauvanaud, Mohamed Kaâniche, Karama Kanoun, Kahina Lazri, and Guthemberg Silvestre. Anomaly detection and diagnosis for cloud services: Practical experiments and lessons learned. J. Syst. Softw., 139:84–106, 2018.

[33] Xiuyao Song, Mingxi Wu, Christopher M. Jermaine, and Sanjay Ranka. Statistical change detection for multi-dimensional data. In KDD, pages 667–676. ACM, 2007.

[34] Charles Truong, Laurent Oudre, and Nicolas Vayatis. Selective review of offline change point detection methods. Signal Process., 167, 2020.

[35] Qingliu Wu, Qingfeng Xiang, Yingxia Shao, Qiyao Luo, and Quanqing Xu. Dbpecker: A graph-based compound anomaly diagnosis system for distributed rdbmss. Proc. VLDB Endow., 18(12):5383–5386, 2025.

[36] Wei Xu, Ling Huang, Armando Fox, David A. Patterson, and Michael I. Jordan. Detecting large-scale system problems by mining console logs. In SOSP, pages 117–132. ACM, 2009.

[37] Zhenkun Yang, Chuanhui Yang, Fusheng Han, Mingqiang Zhuang, Bing Yang, Zhifeng Yang, Xiaojun Cheng, Yuzhong Zhao, Wenhui Shi, Huafeng Xi, Huang Yu, Bin Liu, Yi Pan, Boxue Yin, Junquan Chen, and Quanqing Xu. Oceanbase: A 707 million tpmc distributed relational database system. Proc. VLDB Endow., 15(12):3385–3397, 2022.

[38] Bohan Zhang, Dana Van Aken, Justin Wang, Tao Dai, Shuli Jiang, Jacky Lao, Siyuan Sheng, Andrew Pavlo, and Geoffrey J. Gordon. A demonstration of the ottertune automatic database management system tuning service. Proc. VLDB Endow., 11(12):1910–1913, 2018.

[39] Zhanhao Zhao, Hongyao Zhao, Qiyu Zhuang, Wei Lu, Haixiang Li, Meihui Zhang, Anqun Pan, and Xiaoyong Du. Efficiently supporting multi-level serializability in decentralized database systems. IEEE Transactions on Knowledge and Data Engineering, 35(12):12618–12633, 2023.

[40] Wei Zhou, Chen Lin, Xuanhe Zhou, and Guoliang Li. Breaking it down: An in-depth study of index advisors. Proc. VLDB Endow., 17(10):2405–2418, 2024.

[41] Xuanhe Zhou, Guoliang Li, Zhaoyan Sun, Zhiyuan Liu, Weize Chen, Jianming Wu, Jiesi Liu, Ruohang Feng, and Guoyang Zeng. D-bot: Database diagnosis system using large language models. Proc. VLDB Endow., 17(10):2514–2527, 2024.

[42] Yang Zhou, Zezhou Wang, Sowmya Dharanipragada, and Minlan Yu. Electrode: Accelerating distributed protocols with ebpf. In NSDI, pages 1391–1407. USENIX Association, 2023.

[43] Yang Zhou, Xingyu Xiang, Matthew Kiley, Sowmya Dharanipragada, and Minlan Yu. DINT: fast in-kernel distributed transactions with ebpf. In NSDI, pages 401–417. USENIX Association, 2024.

[44] Xuhang Zhu, Xiu Tang, Sai Wu, Jichen Li, Haobo Wang, Chang Yao, Quanqing Xu, and Gang Chen. Cola: Model collaboration for log-based anomaly detection. Proc. VLDB Endow., 18(11):3979–3987, 2025.

[45] Qiyu Zhuang, Xinyue Shi, Shuang Liu, Wei Lu, Zhanhao Zhao, Yuxing Chen, Tong Li, Anqun Pan, and Xiaoyong Du. Geotp: Latency-aware geo-distributed transaction processing in database middlewares. In ICDE, pages 433–445. IEEE, 2025.